UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CURSO DE CIÊNCIA DA COMPUTAÇÃO
PROGRAMAÇÃO DE COMPUTADORES

LABORATÓRIO 30

ESPECIALIZAÇÃO DE TEMPLATES

EXERCÍCIOS DE REVISÃO

VOCÊ DEVE ACOMPANHAR PARA OBTER INFORMAÇÕES COMPLEMENTARES

1. Transforme a função abaixo em um template para exibir um **vetor** com n elementos de tipo qualquer.

```
void Exibir(int vet[], int n)
{
    cout << "Template A:\n";
    for (int i = 0; i < n; i++)
        cout << vet[i] << ' ';
    cout << endl;
}
```

Teste a função com o código abaixo:

```
int main()
{
    int a = 10;
    int b = 20;
    int c = 30;
    int d = 40;
    int e = 50;

    int nums[5] = { a, b, c, d, e };

    Exibir(nums, 5);
}
```

Teste o programa com outros vetores de tipos diferentes. Tente, por exemplo, exibir os vetores de double e de char mostrados abaixo.

```
double fracs[5] = { 2.1, 3.2, 4.5, 5.6, 6.7 };

char letras[5] = { 'a', 'b', 'c', 'd', 'e' };
```

2. Transforme a função abaixo em um template para exibir um **vetor de ponteiros** para um tipo qualquer.

```
void Exibir(int * vet[], int n)
{
    cout << "Template B:\n";
    for (int i = 0; i < n; i++)
        cout << *vet[i] << ' ';
    cout << endl;
}
```

Teste a função com o código abaixo:

```
int main()
{
    int a = 10;
    int b = 20;
    int c = 30;
    int d = 40;
    int e = 50;

    int* ptrs[5] = { &a, &b, &c, &d, &e };

    Exibir(ptrs, 5);
}
```

3. O que acontece se definirmos os dois templates das questões anteriores no mesmo programa? Teste qual deles será utilizado no programa abaixo.

```
int main()
{
    int a = 10;
    int b = 20;
    int c = 30;
    int d = 40;
    int e = 50;

    int nums[5] = { a, b, c, d, e };
    int* ptrs[5] = { &a, &b, &c, &d, &e };

    Exibir(nums, 5);
    Exibir(ptrs, 5);
}
```

E se removermos o template que trata explicitamente com ponteiros, o programa ainda roda? Se sim, o que será exibido?

Dica: o compilador casa as chamadas de função sempre com o template mais especializado. No caso, a presença explícita do ponteiro no segundo template faz com que ele seja escolhido quando os elementos são endereços.

4. A definição do tipo obtido por **decltype** é feita pelo compilador em vários passos:

   decltype (expressão) variável;

   a. Se a expressão é um identificador, a variável é do mesmo tipo do identificador, incluindo qualificadores, como const.

```
double x = 5.5;
double y = 7.9;

double &num = x;
const double * ptr;

decltype(x)   w;       // w tem tipo double
decltype(num) u;       // u tem tipo double &
decltype(ptr) v;       // v tem tipo const double *
```

   b. Se a expressão contém parênteses, ela é tratada como um l-value e a variável é uma referência para o tipo da expressão.

```
double x = 4.4;

decltype(x) v1;     // v1 tem tipo double
decltype((x)) v2;   // v2 tem tipo double &
```

   c. Se a expressão é uma chamada de função, a variável é do tipo de retorno.

```
float calc(int);
long soma(short, short);

decltype(calc(10)) valor;   // valor tem tipo float
decltype(soma(3,3)) total;  // total tem tipo long
```

   d. Se a expressão é algo diferente dos casos anteriores, a variável é do mesmo tipo da expressão.

```
int j = 3;
int & k = j;
float n = 2.5f;

decltype(j+6) w;       // w tem tipo int
decltype(100L) u;      // u tem tipo long
decltype(k+n) v;       // v tem tipo float
```

EXERCÍCIOS DE FIXAÇÃO

VOCÊ DEVE FAZER OS EXERCÍCIOS PARA FIXAR O CONTEÚDO

1. Construa uma função template que retorne o maior entre dois argumentos. Faça com que ela funcione com o código abaixo.

```
int main()
{
    int a = 10;
    int b = 20;

    float c = 2.5f;
    float d = 4.1f;

    char e = 'M';
    char f = 'N';

    cout << Maior(2, 5) << "\n";
    cout << Maior(a, b) << "\n";
    cout << Maior(c, d) << "\n";
    cout << Maior(e, f) << "\n";
}
```

2. O que acontece se tentarmos usar o template criado na questão anterior com o código abaixo?

```
int main()
{
    cout << Maior(3, 7) << "\n";
    cout << Maior(3.0, 7.0) << "\n";
    cout << Maior(3, 7.0) << "\n";
}
```

   Agora crie uma instanciação explícita do template para o tipo double e tente executar novamente o programa. E se criarmos uma instanciação explícita na chamada da função?

3. A partir do template construído na questão anterior e do registro Caixa mostrado abaixo, construa uma especialização do template que receba dois argumentos do tipo Caixa e retorne aquele com maior volume.

```
struct Caixa {
    float altura;
    float largura;
    float comprimento;
};
```

   Dica: o volume é igual a altura x largura x comprimento da caixa.

4. Que tipos são associados para v1, v2, v3, v4 e v5 no código abaixo?

```
int g(int x);
float m = 5.5f;
float &rm = m;

decltype(m)       v1 = m;
decltype(rm)      v2 = m;
decltype((m))     v3 = m;
decltype(g(100))  v4;
decltype(2.0 * m) v5;
```

5. Escreva um template de função Max( ) que receba como argumento um vetor de elementos de tipo T e a quantidade de elementos no vetor. A função deve retornar o maior elemento do vetor. Teste a função em um programa que utilize um vetor de 5 valores inteiros e um vetor de 5 valores double.

6. Crie uma função template Print com especializações para os seguintes tipos:

    a. **char**: o caractere deve ser exibido entre aspas simples
    b. **char ***: o texto deve ser exibido entre aspas duplas
    c. **bool**: deve exibir “verdadeiro” ou “falso”.

    Outros tipos devem ser exibidos na sua formatação padrão.

7. Transforme o código anterior em uma biblioteca, ou seja, crie um arquivo de cabeçalho (Print.h) contendo apenas o protótipo da função template e um arquivo de código fonte (Print.cpp) contendo as implementações. Faça com que o código abaixo (Main.cpp) compile e rode corretamente.

```
#include <string>
#include "Print.h"

int main()
{
    Print(42);
    Print(3.14);
    Print("Oi Mundo!");
    Print(true);
    Print('A');
}
```

Dica: para que seja possível utilizar a função Print com int e double será necessário criar instanciações explícitas dessas funções no mesmo arquivo .cpp onde se encontram as especializações.

EXERCÍCIOS DE APRENDIZAGEM

VOCÊ DEVE ESCREVER PROGRAMAS PARA REALMENTE APRENDER

1. Crie uma função template bool Iguais(T a, T b) que compara dois valores quaisquer de mesmo tipo.

   Crie especializações para:
   a. **Ponteiros em geral (T *)**: a comparação deve ser feita nos valores apontados, e não nos endereços de memória.
   b. **const char ***: a comparação deve ser feita caractere por caractere (usando strcmp) em vez de comparar endereços.

   Teste as funções com o código abaixo:

```
int main()
{
    int num = 10;
    int val = 10;
    float frac = 7.0f / 2.0f;
    const char name[] = "Template";

    cout << boolalpha;
    cout << Iguais(40, 40) << "\n";
    cout << Iguais(3.5f, frac) << "\n";
    cout << Iguais("Template", name) << "\n";
    cout << Iguais(&num, &val) << "\n";
}
```

2. Crie funções template que retornam a soma e a multiplicação de dois números de tipo qualquer recebidos por parâmetro. Os números podem ter tipos diferentes.

   Teste as funções com o código abaixo:

```
int main()
{
    auto r1 = Add(3, 4.5);      // int + double -> double
    auto r2 = Add('A', 5);      // char + int -> int

    auto r3 = Mul(2.0f, 5);     // float + int -> float
    auto r4 = Mul(1, 2ll);      // int + long long -> long long

    cout << r1 << '\n';         // 7.5
    cout << r2 << '\n';         // 70
    cout << r3 << '\n';         // 10
    cout << r4 << '\n';         // 2
}
```